

Version: final project state after latest gameplay/store/reward updates.

0) How To Use This Document

Use this as:

- Exam speaking script
- Live debugging guide
- "Where do I change this?" map
- Firebase setup and live-demo checklist

1) Project Overview (EN)

Afekoin is an Android app (Kotlin) with Firebase backend.

Main user flow:

1. User signs up / logs in (email + password).
2. User earns AFK coins via mini-games:
 - Quiz
 - Afekliker tap game
 - Liebnitz math-arcade
3. User spends coins in the campus store.
4. User transfers coins to other users by username.
5. User sees transaction history.
6. User can upload profile image.

Firebase services:

- Firebase Authentication
- Firestore
- Firebase Storage

Core architecture choice:

- Activity-based UI with lightweight service/repository objects:
 - `FirebaseWallet.kt` for balance-changing logic
 - `UserRepository.kt` for profile/history/storage operations

(1) סקירת פרויקט (HE)

Afekoin של Firebase Backend עם Kotlin-ב Android היא אפליקציית

זרימת משתמש מרכזית:

1. הרשמה / התחברות (אימייל + סיסמה).
2. צבירת מטבעות AFK דרך משחקים:

- Quiz
- Afekliker
- Liebnitz

3. רכישה בחנות.

4. העברת מטבעות בין משתמשים לפי username.

שירותי Firebase:

- Authentication
- Firestore
- Storage

החלטת ארכיטקטורה:

- Activities מבוסס UI

• לוגיקת כסף מרוכזת ב- `FirebaseWallet.kt`

• גישת נתונים/פרופיל מרוכזת ב- `UserRepository.kt`

2) Top-Level Files (EN)

- `settings.gradle.kts`

Includes module `:app`.

- `build.gradle.kts` (root)

Declares Android + Kotlin plugins and Google services plugin version.

- `app/build.gradle.kts`

App config + Firebase dependencies (BOM/Auth/Firestore/Storage), Coil, coroutines.

- `app/src/main/AndroidManifest.xml`

Registers activities; launcher is `LoginActivity`; internet permission.

- `README.md`

Setup + project summary.

- `firebase.rules.example`, `storage.rules.example`

Example security rules for Firebase console deployment.

- `app/google-services.json` (local, not committed)

Required for real Firebase project connection.

(2) קבצי על (HE)

- `settings.gradle.kts` — הגדרת מודול `app`.

- `build.gradle.kts` — Plugins

- `app/build.gradle.kts` — תלויות וקונפיגורציית אפליקציה

- `AndroidManifest.xml` — רישום מסכים והרשאות

- `README.md` — הוראות הרצה והסבר

- `firebase.rules.example` / `storage.rules.example` — Rules דוגמאות

- `app/google-services.json` — אמיתי Firebase קובץ חיבור לפרויקט

3) Full Kotlin File Map (EN)

3.1 UI Activities / Views

- `app/src/main/java/com/example/mimshak\_final\_afekoin/LoginActivity.kt`
Login screen, remember email, forgot password, enter app.
- `app/src/main/java/com/example/mimshak\_final\_afekoin/SignUpActivity.kt`
Username validation + uniqueness check + auth user creation + initial Firestore user doc.
- `app/src/main/java/com/example/mimshak\_final\_afekoin/MainActivity.kt`
Home screen: profile, balance, daily bonus call, navigation, sign out, profile image upload flow.
- `app/src/main/java/com/example/mimshak\_final\_afekoin/EarnActivity.kt`
Game hub + optional class-time lock.
- `app/src/main/java/com/example/mimshak\_final\_afekoin/AfequizActivity.kt`
Quiz logic, timer, answer checks, scoring, reward save, celebration overlay, results navigation.
- `app/src/main/java/com/example/mimshak\_final\_afekoin/QuizResultActivity.kt`
Shows final quiz score and fresh balance.
- `app/src/main/java/com/example/mimshak\_final\_afekoin/AfeklikerActivity.kt`
Tap game logic, 10-second timer, reward formula, celebration overlay.
- `app/src/main/java/com/example/mimshak\_final\_afekoin/LiebnitzActivity.kt`
Host for custom math-arcade view; computes run reward on game-over; celebration overlay.
- `app/src/main/java/com/example/mimshak\_final\_afekoin/LiebnitzGameView.kt`
Custom rendering/game loop, lane movement, collision, 3 lives, shake on mistakes, increasing speed.
- `app/src/main/java/com/example/mimshak\_final\_afekoin/PayActivity.kt`
Store purchases with item buttons + `FirebaseWallet.charge`.
- `app/src/main/java/com/example/mimshak\_final\_afekoin/TransferActivity.kt`
User-to-user transfer screen.
- `app/src/main/java/com/example/mimshak\_final\_afekoin/HistoryActivity.kt`
Loads and displays transaction history list.
- `app/src/main/java/com/example/mimshak\_final\_afekoin/HistoryAdapter.kt`
RecyclerView adapter for transactions (amount color/sign/time formatting).
- `app/src/main/java/com/example/mimshak\_final\_afekoin/RewardCelebration.kt`
Reusable 2-second bright reward overlay with fireworks animation.
- `app/src/main/java/com/example/mimshak\_final\_afekoin/SoundFx.kt`
Event sound effects.

3.2 Firebase/Data Layer

- `app/src/main/java/com/example/mimshak\_final\_afekoin/firebase/FirebaseWallet.kt`
Balance-changing operations:
 - add credits
 - charge
 - transfer
 - daily bonus
 - transaction logging
- `app/src/main/java/com/example/mimshak\_final\_afekoin/firebase/UserRepository.kt`
Profile read/write, profile photo upload, transaction query.
- `app/src/main/java/com/example/mimshak\_final\_afekoin/firebase/FirestorePaths.kt`

Central path constants (`users`, `transactions`, `profile\_images`).

- `app/src/main/java/com/example/mimshak\_final\_afekoin/data/Profile.kt`

User profile model.

- `app/src/main/java/com/example/mimshak\_final\_afekoin/data/LedgerEntry.kt`

Transaction model.

- `app/src/main/java/com/example/mimshak\_final\_afekoin/QuizQuestion.kt`

Quiz question model.

(3 מפת קבצי HE Kotlin)

חלוקה עיקרית:

- מסכי UI ו-Game Views
- שכבת Firebase/Repository
- מודלים

קבצים חשובים ביותר להבנה בבחינה:

- `FirebaseWallet.kt` — כל שינויי היתרה
- `UserRepository.kt` — גישת נתונים לפרופיל/היסטוריה/תמונה
- `LiebnitzGameView.kt` — לוגיקת משחק מותאמת
- `PayActivity.kt` — רכישות ומחירים
- `TransferActivity.kt` — העברות בין משתמשים
- `AfequizActivity.kt` + `AfeqlikerActivity.kt` + `LiebnitzActivity.kt` — ריוורדים במשחקים
- `RewardCelebration.kt` — חלון 2 שניות עם אנימצית חגיגה

4) Layout XML File Map (EN)

- `app/src/main/res/layout/activity\_login.xml` — login UI
- `app/src/main/res/layout/activity\_sign\_up.xml` — signup UI
- `app/src/main/res/layout/activity\_main.xml` — home dashboard UI
- `app/src/main/res/layout/activity\_earn.xml` — games hub UI
- `app/src/main/res/layout/activity\_afequiz.xml` — quiz UI
- `app/src/main/res/layout/activity\_afequiz\_result.xml` — quiz result UI
- `app/src/main/res/layout/activity\_afeqliker.xml` — tap game UI
- `app/src/main/res/layout/activity\_liebnitz.xml` — Liebnitz game container + HUD
- `app/src/main/res/layout/activity\_pay.xml` — college store UI, vertical scrollbar, extended items
- `app/src/main/res/layout/activity\_transfer.xml` — transfer UI
- `app/src/main/res/layout/activity\_history.xml` — history list UI
- `app/src/main/res/layout/item\_transaction.xml` — single history row

Assets / resources

- `app/src/main/assets/questions.json` — quiz content
- `app/src/main/res/values/strings.xml` — text resources
- `app/src/main/res/values/colors.xml` — palette
- `app/src/main/res/values/themes.xml` — app theme

(4 מפת קבצי HE Layout)

- Login / Signup / Main / Earn / Quiz / QuizResult / Afekliker / Liebnitz / Pay / Transfer / History
- עבור שורת היסטוריה בודדת `item\_transaction.xml`
- לשאלות `questions.json` Quiz
- לשינוי טקסטים מהר `strings.xml`

5) Major Coding Decisions (EN)

1. **Centralized wallet logic** in `FirebaseWallet.kt`
Why: prevents duplicated money logic and reduces inconsistency.
2. **Firestore transactions** for balance updates
Why: safer for concurrent updates (spend/transfer/earn).
3. **Server-refresh reads** on balance-sensitive screens
Why: avoids stale cache after operations.
4. **Username-based transfers**
Why: easier user experience than sharing UID.
5. **Quiz from local JSON asset**
Why: reliable demo/offline-friendly content loading.
6. **Custom game view for Liebnitz**
Why: full control over performance, feel, and animation.
7. **Reusable reward overlay** (`RewardCelebration.kt`)
Why: one consistent post-game UX across all games.
8. **Rules and config outside git** (`google-services.json`)
Why: avoid committing sensitive project config.

(5) החלטות קוד מרכזיות (HE)

1. מרכז את כל לוגיקת הכסף `FirebaseWallet.kt`.
2. שימוש ב-Firebase Transaction לעדכוני יתרה.
3. קריאות שרת מעודכנות למסכים רגישים ליתרה.
4. העברה לפי username לנוחות משתמש.
5. שאלות Quiz מקובץ מקומי יציב לדמו.
6. מותאם אישית לשליטה מלאה במשחק `LiebnitzGameView`.
7. אחיד לכל המשחקים `RewardCelebration`.
8. קבצי Firebase רגישים לא נכנסים ל-Git.

6) Latest Final Updates (EN)

Gameplay updates

- Liebnitz now has:
 - slightly faster pace
 - 3 lives
 - shake animation on mistake

- no visual reveal of correct pod

Store updates

- `activity\_pay.xml`:
 - subtle vertical scrollbar
 - 4 new items
- `PayActivity.kt` new purchase handlers:
 - Daily Parking Pass
 - Lab Kit Refill
 - Power Bank
 - Wireless Mouse

Post-game celebration

- `RewardCelebration.kt` added and called from:
 - `AfequizActivity.kt`
 - `AfeklikerActivity.kt`
 - `LiebnitzActivity.kt`

Behavior:

- after game reward is determined, show bright animated 2-second reward window.

(6) עדכוני גרסה אחרונים (HE)

- מהיר יותר, כולל 3 חיים ושייק בטעות Liebnitz.
- אין הדגשה צבעונית לתשובה נכונה ב-Liebnitz.
- לחנות נוספו 4 פריטים + scrollbar צדדי עדין.
- לכל משחק נוסף חלון חגיגה של 2 שניות אחרי תגמול.

7) Live Demo: Where To Change Things Quickly (EN)

If tester asks: "Change reward amount"

- Quiz reward: `AfequizActivity.kt` (`reward = score.toDouble()`)
- Afekliker reward: `AfeklikerActivity.kt` (`min(2.0, taps \* 0.02)`)
- Liebnitz reward: `LiebnitzActivity.kt` (`min(5.0, finalScore \* 0.1)`)
- Shared wallet updates: `FirebaseWallet.kt`

If tester asks: "Change store prices"

- Main logic: `PayActivity.kt` (`buy(amount, label)` calls)
- UI labels: `activity\_pay.xml` button text

If tester asks: "Change game speed/lives"

- `LiebnitzGameView.kt`
 - update tick delay
 - update `baseSpeed`
 - update lives init / reset

If tester asks: "Change what shows after game"

- `RewardCelebration.kt`
 - text
 - duration (`2000L`)
 - colors/animation style

If tester asks: "Change transfer behavior"

- UI validation: ``TransferActivity.kt``
- transfer transaction logic: ``FirebaseWallet.transferToUsername``

If tester asks: "Change signup rules"

- ``SignUpActivity.kt`` username regex and validation

If tester asks: "Change daily bonus"

- ``FirebaseWallet.checkAndGrantDailyBonus``

(7) איפה משנים בזמן דמו (HE)

- תגמולי משחקים: ``AfequizActivity.kt``, ``AfeklikerActivity.kt``, ``LiebnitzActivity.kt``
- מחירי חנות: ``PayActivity.kt`` + ``activity_pay.xml``
- מהירות/חיים ב-``Liebnitz``: ``LiebnitzGameView.kt``
- חלון חגיגה: ``RewardCelebration.kt``
- לוגיקת העברה: ``TransferActivity.kt`` + ``FirebaseWallet.kt``
- כללי הרשמה: ``SignUpActivity.kt``
- בונוס יומי: ``FirebaseWallet.kt``

8) Firebase Practical Guide (EN)

8.1 One-time setup

1. Create/select Firebase project.
2. Add Android app with package:
 - ``com.example.mimshak_final_afekoin``
3. Download ``google-services.json`` to:
 - ``app/google-services.json``
4. Enable Email/Password auth in Authentication.
5. Create Firestore database.
6. Enable Storage.
7. Publish Firestore and Storage rules.

8.2 App integration in this project

- Plugin: ``com.google.gms.google-services``
- Dependencies: Firebase BOM + Auth + Firestore + Storage
- Auth usage: login/signup/reset flows
- Firestore usage:
 - ``users`` docs for profiles/balance
 - ``transactions`` docs for ledger
- Storage usage:
 - ``profile_images/{uid}.jpg``

8.3 Live verification checklist

- New signup appears in Firebase Authentication.
- New user doc appears in Firestore ``users``.
- Playing a game changes ``balance``.
- Transfer creates two transaction effects.
- Upload image appears in Storage and updates ``photoUrl``.

8.4 Common Firebase errors and fixes

`PERMISSION\_DENIED`

- Usually rules mismatch.
- Confirm project and published rules.
- Confirm user is authenticated if rule requires it.

Recipient not found

- Username mismatch vs stored lowercase username.

Writes succeed but UI stale

- Force server read where needed (`forceServer = true`).

Build fails related to Firebase/Gradle

- Ensure JDK is 11+ (recommended 17).
- Ensure `app/google-services.json` exists and package matches app ID.

(8) מדריך Firebase מעשי (HE)

התקנה ראשונית

1. לבחור פרויקט Firebase.
2. להוסיף app Android עם package נכון.
3. לשים `google-services.json` בתוך `app`.
4. להפעיל Email/Password Auth.
5. להפעיל Firestore ו-Storage.
6. לפרסם Rules.

בדיקות בזמן אמת

- משתמש חדש מופיע ב-Authentication.
- מסמך משתמש מופיע ב-`users`.
- אחרי משחק היתרה משתנה.
- אחרי העברה יש עדכון ב-`transactions`.
- העלאת תמונה נכנסת ל-Storage ומתעדכנת ב-`photoUrl`.

תקלות נפוצות

- הרשאות Rules בעיית => `PERMISSION\_DENIED` שם משתמש לא נמצא => בדוק lowercase והתאמה מלאה.
- יתרה לא מתעדכנת במסך => צריך קריאת שרת מעודכנת.
- כשל JDK 11 => build + ו-`google-services.json` תקין.

9) Full Live Exam Script (EN)

1. Open Android Studio project.
2. Open Firebase console tabs:
 - Authentication
 - Firestore (`users`, `transactions`)
 - Storage
3. Build and run app.
4. Sign up a new demo user.
5. Show user appears in Authentication + Firestore.
6. Play one game and show 2-second celebration window.

7. Show balance increased.
8. Open history and show transaction.
9. Open store, buy one item, show balance decrease.
10. Transfer small amount to another user.
11. Show transfer effect in Firestore.
12. Sign out and sign back in to prove persistence.

(9) תסריט בחינה מלא (HE)

1. לפתוח פרויקט ב-Android Studio.
2. לפתוח קונסול (Firebase (Auth, Firestore, Storage).
3. להריץ את האפליקציה.
4. לבצע הרשמה למשתמש חדש.
5. להראות הופעה ב-Auth + Firestore.
6. לשחק משחק ולהראות חלון חגיגה 2 שניות.
7. להראות עלייה ביתרה.
8. להציג History.
9. לבצע רכישה בחנות.
10. לבצע Transfer למשתמש אחר.
11. להציג עדכונים ב-Firebase.
12. להתנתק ולהתחבר שוב להוכחת שמירה.

10) Build/Compile Checklist (EN + HE)

- [] Gradle JDK set to 11+ (prefer 17)
- [] `app/google-services.json` exists
- [] Gradle sync successful
- [] App runs on emulator/device
- [] Signup/login flow tested
- [] Quiz / Afekliker / Liebnitz tested
- [] Store purchase tested
- [] Transfer tested with second account
- [] Firebase tabs ready for demo
- [] Stable internet

11) Security and Professional Notes (EN)

- Do not expose secrets during recording.
- Use clearly labeled demo users.
- Keep Firebase rules demo-safe but understand production needs stronger rules.
- Keep business-critical balance changes centralized in wallet logic.

(11) הערות מקצועיות (HE)

- לא לחשוף סודות/API keys בצילום.
- להשתמש במשתמשי דמו מסודרים.
- להבין ש-Rules לדמו יכולים להיות פתוחים יותר מפרודקשן.

12) Quick "If They Ask Me..." Answer Bank (EN)

- "Where is transfer atomic?"

`FirebaseWallet.transferToUsername` uses Firestore transaction updating sender+recipient together.

- "Where to add a new store item?"

Add button+card in `activity\_pay.xml`, add click listener in `PayActivity.kt`.

- "Where to change daily bonus?"

`FirebaseWallet.checkAndGrantDailyBonus`.

- "Where to adjust game difficulty?"

`LiebnitzGameView.kt` speed/lives/shake constants.

- "Where to change quiz questions?"

`app/src/main/assets/questions.json`.

- "How do you confirm backend works?"

Show immediate Auth + Firestore + Storage updates from app actions.

(12) בנק תשובות קצר (HE)

- איפה ההעברה אטומית?

`FirebaseWallet.transferToUsername` עם Firestore Transaction.

- איפה מוסיפים פריט לחנות?

`activity\_pay.xml` + `PayActivity.kt`.

- איפה משנים בונוס יומי?

`FirebaseWallet.checkAndGrantDailyBonus`.

- איפה משנים קושי משחק?

`LiebnitzGameView.kt`.

- איפה משנים שאלות Quiz?

`assets/questions.json`.

- איך מוכיחים ש-Firebase עובד?

מראים שינוי בזמן אמת ב-Auth/Firestore/Storage.

13) Final Oral Closing (EN)

"This app is a Kotlin Android client using Firebase Authentication, Firestore, and Storage.

The architecture keeps money-critical logic in `FirebaseWallet` and data operations in `UserRepository`, while Activities handle UI flow.

Users can register, earn, spend, transfer, and track history, and I can change behavior live by editing the relevant feature file and immediately verifying results in Firebase."

(13) משפט סיום קצר (HE)

"האפליקציה בנויה ב-Kotlin על Android עם Firebase Auth, Firestore ו-Storage. לוגיקת הכסף מרוכזת ב-FirebaseWallet, גישת הנתונים ב-UserRepository, וה-Activities מנהלים את ה-UI. אפשר לשנות התנהגות בזמן אמת לפי קובץ פיצ'ר ולהוכיח הכל בלייב מול Firebase."